

Course: Full Stack Web Development

Course Code: 23CM4121

Task No.: 3

Student Name: M. Vanisree

Roll No.: A24126552125

Date: 25-08-2026

Title

Develop To-Do list application

Aim

To develop a simple **To-Do List application** using HTML, CSS, JavaScript, DOM manipulation, and event listeners. The application allows users to add tasks, mark tasks as completed, delete tasks, and displays an appropriate message when no tasks are available.

OBJECTIVE

The objectives of this task are:

- To understand the basics of HTML, CSS, and JavaScript.
 - To create the structure of a To-Do List application using HTML.
 - To style the webpage using CSS properties.
 - To manipulate HTML elements dynamically using JavaScript.
 - To add new tasks dynamically to the webpage.
 - To implement event listeners for button click events.
 - To mark tasks as completed using CSS classes.
 - To delete tasks dynamically from the DOM.
 - To display an appropriate message when no tasks are available.
 - To understand interactive webpage development using JavaScript.
-

THEORY

The **To-Do List application** is an interactive web application developed using **HTML, CSS, and JavaScript**.

HTML is used to create the basic structure of the webpage, including a text input field for entering tasks, an **Add Task** button, a task list area, and a message displaying "**No tasks available**" when the list is empty.

CSS is used to style the application by applying background colors, spacing, borders, rounded corners, shadows, and layouts. A CSS class named `.completed` is used to visually indicate that a task has been completed.

JavaScript is used for **DOM manipulation and event handling**. When the user enters a task and clicks the **Add Task** button, a new task element is dynamically created using `createElement()`.

Each task contains a **Complete** button and a **Delete** button. The Complete button uses `classList.toggle()` to add or remove the `completed` class. The Delete button removes the corresponding task from the webpage using the `remove()` method.

When there are no tasks in the list, the "**No tasks available**" message is displayed again.

The basic flow of the application is:

User Input → Add Task → Create Task Element → Add to DOM → Complete/Delete Task → Update Task List

ALGORITHM / PROCEDURE

1. Create an HTML file for the To-Do List application.
2. Add a heading titled **My To-Do List**.
3. Create a text input field for entering tasks.
4. Create an **Add Task** button.
5. Create a task list container.
6. Add a "**No tasks available**" message inside the task list.
7. Create a CSS file to style the webpage.
8. Style the input field, button, task list, and task items.
9. Define a `.completed` CSS class to show completed tasks using a line-through effect.
10. Create a JavaScript file.
11. Select the required HTML elements using `getElementById()`.
12. Add a click event listener to the **Add Task** button.
13. Retrieve and trim the task entered by the user.
14. If the input is empty, display an alert message.
15. Otherwise, dynamically create a task item using `createElement()`.

16. Create **Complete** and **Delete** buttons for the task.
17. Add a click event listener to the Complete button.
18. Toggle the completed class when the Complete button is clicked.
19. Add a click event listener to the Delete button.
20. Remove the task when the Delete button is clicked.
21. Display the "**No tasks available**" message when all tasks are deleted.
22. Clear the input field after adding a task.
23. Open the webpage in a browser.
24. Perform multiple test cases and verify the actual output.

PROGRAM

HTML Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="./style.css">
</head>
<body>
  <h1>My To-Do List</h1>
  <div class="container">
    <label for="task">Enter the Task:</label>
    <input type="text" name="" id="task" placeholder="Please enter your task">

    <br><br>

    <button id="addBtn">Add Task</button>

  </div><br><br>
  <div id="taskList">
    <p id="emptyMessage">No tasks available</p>
  </div>
  <script src="./script.js"></script>
</body>
</html>
```

CSS Code:

```
body{
  background-color: blueviolet;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  gap: 10px;
}
.container{
  background-color: white;
  padding: 20px;
  border-radius: 8px;
  font-size: 1.2rem;
  box-shadow: 10px 20px 30px rgba(0, 0, 0, 0.15);
}
#task{
  border-radius: 4px;
  border: none;
  background-color: rgb(183, 133, 231);
  height: 20px;
  padding: 5px;
}
#task::placeholder{
  color: white;
}
#addBtn{
  margin-left: 100px;
  cursor: pointer;
  padding: 10px;
  background-color: burlywood;
  border: none;
  border-radius: 4px;
}
#taskList{
  display: flex;
  flex-direction: column;
  gap: 10px;
  background-color: burlywood;
  padding: 10px;
  border-radius: 10px;
  box-shadow: 10px 20px 30px rgba(0, 0, 0, 0.15);
}
.task-item {
  background-color: white;
  padding: 12px 15px;
  border-radius: 5px;

  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;

  min-width: 300px;
}
.task-item button {
  padding: 4px 10px;
  cursor: pointer;
  border: none;
  border-radius: 4px;
  background-color: burlywood;
}
.completed{
  text-decoration: line-through;
  opacity: 0.6;
}
```

Javascript Code:

```
const task = document.getElementById("task");
const addBtn = document.getElementById("addBtn");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");

addBtn.addEventListener("click", function(){
  const taskValue = task.value.trim()

  if(taskValue === ""){
    alert("Please Enter the task")
  }
  else{
    const taskItem = document.createElement("div")
    taskItem.textContent = taskValue;
    taskItem.classList.add("task-item");

    const completeBtn = document.createElement("button");
    completeBtn.textContent = "Complete";

    const deleteBtn = document.createElement("button");
    deleteBtn.textContent = "Delete";

    completeBtn.addEventListener("click", function(){
      taskItem.classList.toggle("completed");
    });

    deleteBtn.addEventListener("click", function(){
      taskItem.remove();
    });

    if(taskList.children.length === 1){
      emptyMessage.style.display = "block";
    }
  }

  taskItem.appendChild(completeBtn);
  taskItem.appendChild(deleteBtn)

  taskList.appendChild(taskItem);

  emptyMessage.style.display = "none";

  task.value = "";
})
});
```

Code Explanation

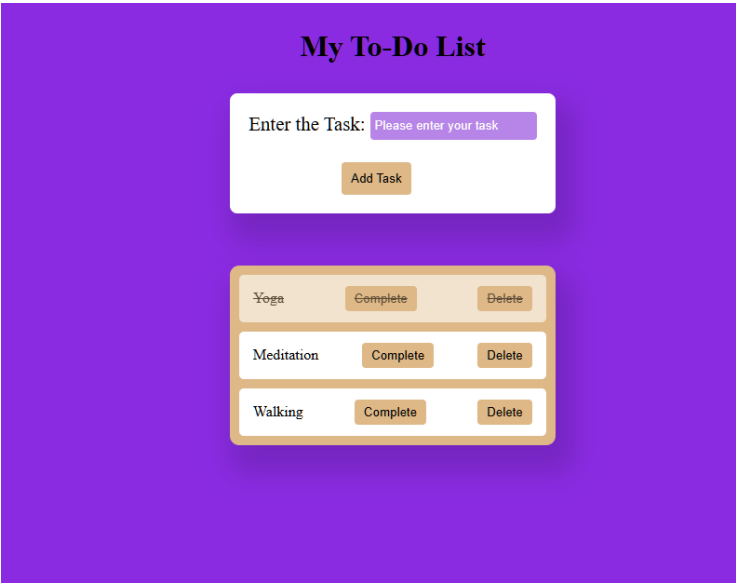
Code	Explanation
getElementById()	Fetches an HTML element using its ID.
addEventListener()	Registers event listeners for DOM elements, handles events such as button clicks.
value	Retrieves the value entered in the input field.
trim()	Removes unnecessary spaces from the input.
createElement()	Creates a new HTML element dynamically.

Content	or retrieves the text content of an element.
classList.add()	adds a CSS class to an element.
classList.toggle()	adds or removes a CSS class dynamically.
appendChild()	adds a newly created element to another element.
remove()	removes an element from the webpage.
element.style.display	controls whether an element is visible or hidden.
task.completed	applies line-through and reduced opacity to completed tasks.
task.value === "")	checks whether the task input is empty.
showMessage()	displays a message when the user tries to add an empty task.

TEST CASES AND EXECUTION DATA

Case ID	Component Tested	Action / Input	Expected Result	Actual Result	Status
001	Add Task	Enter "Complete assignment" and click Add Task	Task should be added to the task list	Task added successfully	Pass
002	Empty Task Validation	Leave the input field empty and click Add Task	Error message "Please enter the task" should be displayed	Error displayed successfully	Pass
003	Complete Task	Click a task and click Complete	Task should be marked as completed with line-through effect	Task marked as completed successfully	Pass
004	Delete Task	Click a task and click Delete	Selected task should be removed from the list	Task deleted successfully	Pass
005	Empty Task Validation	Click Complete all tasks	Message "No tasks available" should be displayed	Message displayed successfully	Pass

OUTPUT:



RESULT

Thus, a To-Do List application was successfully developed using HTML, CSS, and JavaScript. DOM manipulation and event listeners were used to add, complete, and delete tasks dynamically. Empty task validation and the "No tasks available" message were also implemented successfully. The application was tested using multiple test cases and the expected results were verified.